Chapter 2

Background

To improve the reliability of developed software, testing technique is widely used in the industry of software. However, there is an important problem that need to be solved in the verification of software, namely concurrent program verification. As we know, concurrency errors are particularly hard to find by testing technique, because the errors are difficult to reproduce. Model checking technique, an efficient and exhaustive verification technique, can solve the problem. As shown in Fig. 2.1, the key principle of model checking is: let M be a state transition graph of a system (i.e., Kripke structure or finite state machine), let f be a given property (i.e., assertion or temporal logic formula), explore all states s of M to determine whether the given property f is true in the state transition graph M (i.e., $M, s \models f$). If the given property f is false in the state transition graph M, the trace in the state transition graph M can be considered as a witness for revealing the given

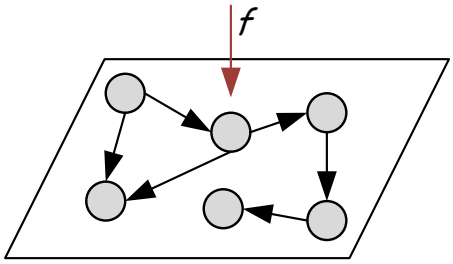

Figure 2.1: The principle of model checking

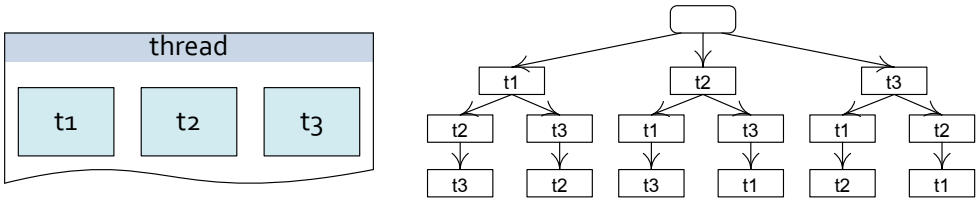

Figure 2.2: The model for a SystemC multi-threaded software

property f. In model checking, the trace is usually named counterexample. Based on the counterexample, the errors can be easily reproduced.

In this chapter, the model checking for concurrent software and linear temporal logic (LTL) will be firstly introduced, and then the advanced bounded model checking (BMC) technique will be discussed based on several examples.

2.1 Model Checking

2.1.1 Model Checking for Concurrent Software

Concurrent software usually consists of multiple threads. To concurrently execute multiple threads on one CPU, a scheduler is used to dispatch threads. There have been many concurrent software standards such SystemC standard, POSIX standard and ANSI-C standard that have been widely used in the industry of software. In these concurrent software standards, which thread within software is to be run is arbitrarily determined by scheduler. In other words, the running thread is stochastic when the multi-threaded software runs on the machine. If testing technique is used to check such software, it is difficult to reproduce an error track because of the non-deterministic scheduling. In contrast to testing technique, model checking can solve the problem based on the reported counterexample.

The key idea of model checking is: firstly construct a model to describe the executions of a target system, and then search the constructed model with given property. To verify concurrent software, a tree as a model is usually constructed for demonstrating all of the

possible interleavings of threads. E.g., as shown in Fig. 2.2, there is a SystemC multi-threaded software, and the multi-threaded software consists of three threads. In model checking, a tree shown in the right side of Fig. 2.2 as a model will be constructed to exhaustively describe all of the possible executions of the multi-threaded software. When the model of concurrent software is constructed, there existing many checking techniques that can be applied to search the constructed model for checking whether the constructed model satisfies the given property or not, such as explicit state checking technique [23], binary decision diagrams (BDD) technique [6][45], symbolic checking technique [21] and bounded model checking technique [24].

Currently, based on the model checking technique, there are many model checkers that have been established based on the different model checking techniques, and these established model checkers have been successfully applied to verify multi-threaded software that conforms to different concurrent software standards. E.g., Bell Labs developed a model checker named `spin` [26] based on the explicit state model checking technique. For the ANSI-C standard, a bounded model checking based model checker named `esbmc` [31] has been implemented in the university of Southampton. In addition, in the group of Cimatti, a model checker named `kratos` [12] has been established for SystemC multi-threaded software. There are also several famous model checkers, such as symbolic model checker `SMV` [7], bounded model checker `cbmc` [16], etc.

2.1.2 Liner Temporal Logic (LTL) Specification

In the verification of software, assertion as a common artifice is usually inserted into the model of target software to judge whether the target software models the specification. However, it is hard to use assertion to exhaustively check whole model of concurrent software, because the insertion position of assertion is confoundedly difficult to be determined, caused by concurrency. Linear Temporary Logic (LTL) is an instance of modal logic, instead of assertion, often used to specify properties in the verification of concurrent software, e.g., it can be used to specify a property for judging whether the value of a vari-

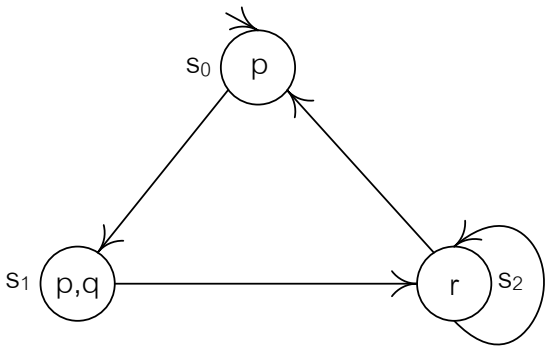

Figure 2.3: An example for Kripke structure

able will become a specified value in the future. In this part, based on Kripke structure [14] and paths of Kripke structure, we will firstly introduce the semantics of LTL, and then show several examples to explain some general properties specified in LTL.

Definition 1 *A Kripke structure M is a tuple $M = (S, I, R, L)$. $S = \{s_1, s_2, ..., s_n\}$ is the finite set of states. $I \subseteq S$ is the set of initial states. $R \subseteq S \times S$ is the transition relation, where $\forall s \in S, \exists s' \in S$ such that $(s, s') \in R$. $L : S \to 2^{AP}$ is labelling function from states to the power set of atomic propositions AP, where for a state $s \in S$, the set $L(s)$ is made of the atomic propositions that hold in s.*

For example, a kripke structure is shown in Fig. 2.1. In the shown example, $S = \{s_0, s_1, s_2\}$, and p, q, r are the atomic propositions.

Definition 2 *A path π of a Kripke structure M is an infinite sequence $s_0, s_1, \cdots$ of states, where $\forall i, \langle s_i, s_{i+1} \rangle \in R$. $\pi^i = s_i, s_{i+1}, \cdots$ is the subpath of π starting from state s_i, and $\pi(i)$ denotes the i-th state s_i of path π. If the starting state $\pi(0)$ of a path π is an initial state (i.e., $\pi(0) \in I$), then we say that the path π is initialized.*

For example, as shown in Fig. 2.3, the path $\pi = (s_0, s_1, s_2, s_2, \cdots)$ is an initialized path in the shown Kripke structure, where $\pi(0)$ is the initial state s_0.

In general, LTL inherits propositional variables (in mathematical logic, a propositional variable is the variable which can either be true or false) and logic operators such

as negation $\neg$, conjunction $\wedge$, disjunction $\vee$ and implication $\rightarrow$, etc. In addition to the propositional variables and logic connectives, LTL formula holds several temporal operators, such as the next time operator $\mathtt{X}$, the global operator $\mathtt{G}$, the liveness operator $\mathtt{F}$, the binary temporal operators Until ($\mathtt{U}$) and Release ($\mathtt{R}$). The syntax of LTL over atomic propositions AP with $p \in AP$ is stated below,

$$\varphi ::= true|false|p|\varphi_1 \wedge \varphi_2|\varphi_1 \vee \varphi_2|\varphi \rightarrow \varphi|\neg\varphi|\mathtt{X}\varphi|\mathtt{F}\varphi|\mathtt{G}\varphi|\varphi\mathtt{U}\varphi|\varphi\mathtt{R}\varphi$$

For example, for the given example shown in Fig. 2.3, we can specify a LTL formula $\varphi = \mathtt{F}(p \wedge q)$ to verify the system.

Definition 3 *Let π be a path in Kripke structure M, φ be a LTL formula. The LTL formula φ is held by the path π iff $M, \pi \models \varphi$, where*

$M, \pi \models true \qquad M, \pi \not\models false$

$M, \pi \models p \qquad iff \quad p \in L(\pi(0))$

$M, \pi \models \neg\varphi \qquad iff \quad M, \pi \not\models \varphi$

$M, \pi \models \varphi_1 \wedge \varphi_2 \quad iff \quad M, \pi \models \varphi_1 \text{ and } M, \pi \models \varphi_2$

$M, \pi \models \varphi_1 \vee \varphi_2 \quad iff \quad M, \pi \models \varphi_1 \text{ or } M, \pi \models \varphi_2$

$M, \pi \models \varphi_1 \rightarrow \varphi_2 \quad iff \quad M, \pi \not\models \varphi_1 \text{ or } M, \pi \models \varphi_2$

$M, \pi \models \mathtt{G}\varphi \qquad iff \quad \text{for all paths } \pi^i \text{ for all } i \geq 0,\ M, \pi^i \models \varphi$

$M, \pi \models \mathtt{F}\varphi \qquad iff \quad \text{for some path } \pi^i \text{ for some } i \geq 0,\ M, \pi^i \models \varphi$

$M, \pi \models \mathtt{X}\varphi \qquad iff \quad M, \pi^1 \models \varphi$

$M, \pi \models \varphi_1\mathtt{U}\varphi_2 \qquad iff \quad \text{exists } i \geq 0 \text{ such that } M, \pi^i \models \varphi_2 \text{ and for all } j < i,\ M, \pi^j \models \varphi_1$

$M, \pi \models \varphi_1\mathtt{R}\varphi_2 \qquad iff \quad \text{exists } i \geq 0 \text{ such that } M, \pi^i \models \varphi_1 \text{ and for all } j \leq i,\ M, \pi^j \models \varphi_2 \text{ or for all } i \geq 0\ M, \pi^i \models \varphi_2$

For the example shown in Fig. 2.3, according to the semantics of LTL stated in definition 3, we can easily find that the system holds the LTL formula $\varphi = \mathtt{F}(p \wedge q)$.

2.2 Bounded Model Checking

The original motivation of bounded model checking was to leverage the success of satisfiability (SAT) [39][41] in solving Boolean formulas to model checking. Compared to explicit state model checking technique and BDD-based model checking technique [6][45], it offers the advantage of handling the verification of large state spaces. The main idea of BMC is to avoid the full state space generation and look for witnesses of an existential specification on the suitable subsets of the full model under the given bound. Once a sub-model is selected, the considered sub-model will be translated into propositional formulae. Then, the propositional formulae and the negative form of given specification (given property) will be solved by a specialized SAT solver. If the verification result is true, it means that the sub-model does not satisfy the given specification, a counterexample will be report by the SAT solver. Otherwise, a larger sub-model will be selected and the whole procedure will be run again until the whole model is verified.

2.2.1 Transition system with Bound

Bounded model checking consists of a transition system M, a given property f and a user-supplied bound k ($k \geq 0$). The transition system M is a conjunctive normal formula (CNF), used to model the target system. The given property can be stated as a LTL formula. The user-supplied bound k is used to limit the verification space. If given a system M, a bound k and a property f, we can use the propositional formula shown in definition 4 to carry out verification based on the SAT solver.

Definition 4 $[\![M, \neg f]\!]_k := I(s_0) \wedge (\bigwedge T(s_k, s_{k+1})) \wedge [\![\neg f]\!]_k$

Where, $I(s_0)$ is the initial function used to initialize the values of variables declared in the system, $T(s_k, s_{k+1})$ is the transition relation.

For example, as shown in Fig. 2.4, for a 2-bit counter which is taken from paper [24], if the bound k is set to 2, we can use the following transition system $[\![M]\!]_2$ to model the

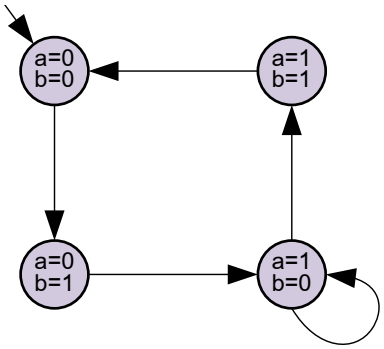

Figure 2.4: A 2-bit counter

system, where the initial function $I(s_0)$ is $(a_0 = 0 \wedge b_0 = 0)$.

$$[\![M]\!]_2 := I(s_0) \wedge (a_0 = 0 \wedge b_0 = 0) \wedge (a_1 = 0 \wedge b_1 = 1) \wedge (a_2 = 1 \wedge b_2 = 0)$$

2.2.2 LTL Property in Bounded Model Checking

Bounded model checking supports the property specified in LTL formula in the verification. The semantics of LTL property f in bounded model checking are shown in definition 5.

Definition 5 *Let $k \geq 0$, and let π be a path in transition system M. Then a LTL formula f is valid along π with bound k ($\pi \models_k f$) iff $\pi \models_k^0 f$, where*

$\pi \models_k^i p$ *iff* $p \in L(\pi(i))$

$\pi \models_k^i \neg p$ *iff* $p \notin L(\pi(i))$

$\pi \models_k^i f \wedge g$ *iff* $\pi \models_k^i f$ *and* $\pi \models_k^i g$

$\pi \models_k^i f \vee g$ *iff* $\pi \models_k^i f$ *or* $\pi \models_k^i g$

$\pi \models_k^i \mathtt{G}f$ *is always false*

$\pi \models_k^i \mathtt{F}f$ *iff* $\exists j, i \leq j \leq k . \pi \models_k^i f$

$\pi \models_k^i \mathtt{X}f$ *iff* $i \leq k$ *and* $\pi \models_k^{i+1} f$

$\pi \models_k^i f \mathtt{U} g$ *iff* $\exists j, i \leq j \leq k . \pi \models_k^j g$ *and* $\forall n, i \leq n \leq j . \pi \models_k^n f$

$\pi \models_k^i f \mathtt{R} g$ *iff* $\exists j, i \leq j \leq k . \pi \models_k^j f$ *and* $\forall n, i \leq n \leq j . \pi \models_k^n g$

Based on the semantics shown in definition 5, the following functions can be used to translate a LTL property f into the verification expression with bound k, which have been stated in paper [14]. In the translation function, $i \leq k$, $succ(i) := i + 1$ and $[\![f]\!]_k^{k+1} := 0$.

$$[\![p]\!]_k^i := p(s_i)$$

$$[\![\neg p]\!]_k^i := \neg p(s_i)$$

$$[\![f \wedge g]\!]_k^i := [\![f]\!]_k^i \wedge [\![g]\!]_k^i$$

$$[\![f \vee g]\!]_k^i := [\![f]\!]_k^i \vee [\![g]\!]_k^i$$

$$[\![\mathtt{G}h]\!]_k^i := [\![h]\!]_k^i \wedge [\![\mathtt{G}h]\!]_k^{succ(i)}$$

$$[\![\mathtt{F}h]\!]_k^i := [\![h]\!]_k^i \vee [\![\mathtt{F}h]\!]_k^{succ(i)}$$

$$[\![\mathtt{X}h]\!]_k^i := [\![\mathtt{X}h]\!]_k^{succ(i)}$$

$$[\![h\mathtt{U}g]\!]_k^i := [\![g]\!]_k^i \vee ([\![h]\!]_k^i \wedge [\![h\mathtt{U}g]\!]_k^{succ(i)})$$

$$[\![h\mathtt{R}g]\!]_k^i := [\![g]\!]_k^i \wedge ([\![h]\!]_k^i \vee [\![h\mathtt{R}g]\!]_k^{succ(i)})$$

For example, for the given example shown in Fig. 2.4, if we given a LTL property $f = \mathtt{F}(a = 1)$ and set bound k to 2, the below expression can be used to verify the system with the given LTL property f.

$$[\![M, \neg f]\!]_2 := (a_0 = 0 \wedge b_0 = 0) \wedge (a_1 = 0 \wedge b_1 = 1) \wedge (a_2 = 1 \wedge b_2 = 0) \wedge (\neg(a_0 = 1 \vee a_1 = 1 \vee a_2 = 1))$$

2.2.3 Satisfiability Modulo Theories (SMT)

In bounded model checking, a SAT solver [8][28][40] is usually employed to check whether the constructed transition system M satisfies the given property f (i.e., $M \models_k f$). Although the SAT-based BMC [38] is more efficient and scalable than explicit state model checking technique and BDD model checking technique, the checking ability of SAT-based BMC is still limited by the solving performance of SAT solver when checking a complex

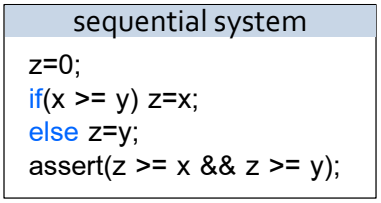

Figure 2.5: A sequential C system

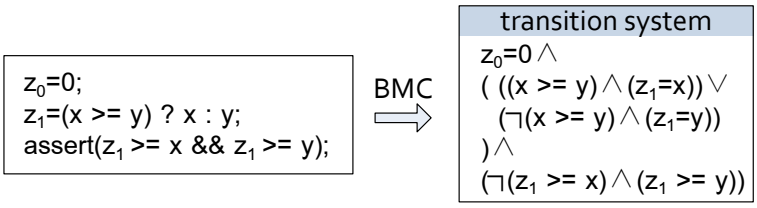

Figure 2.6: The transition system for the given sequential C system

system which holds a large number of states. Fortunately, an important research [11] of checking a transition system with satisfiability modulo theory (SMT) solver [17] instead of SAT solver shows that SMT not only can check more conditions of the transition system within shorter time than SAT solver, but also supports arithmetic, bitvectors, tuples, arrays, and other decidable first-order theories which allow us to more conveniently translate the transition system M into SMT solver.

Currently, based on the satisfiability modulo theory, there exist many SMT solvers that have been implemented by researchers and software company. E.g., Microsoft Research group has developed a SMT solver named Z3 [35], which has been widely used in the research field and industry of software. Yices [20] developed by SRI is also a famous and efficient SMT solver. In addition, there are other SMT solvers that are using in the academic institution, such as MiniSmt [27], Mistral [29] and OpenSMT [15].

2.2.4 SMT-based BMC for Sequential Software and Concurrent Software

SMT-based BMC has been widely applied in the verification of sequential software and concurrent software. The key work of using BMC to verify a system is how to construct

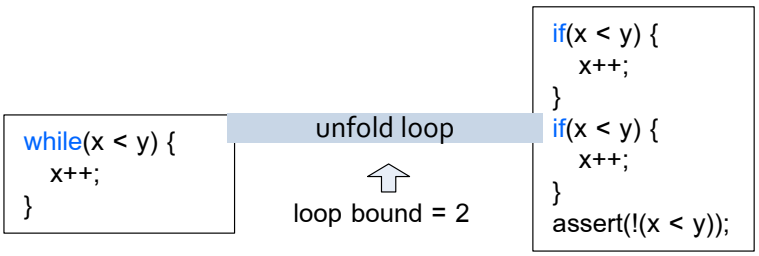

Figure 2.7: BMC for loop

a transition system M. Firstly, let us consider a sequential C system which is shown in Fig. 2.5. In the system, there is a branch that is used to return the maximum value of variables x and y to variable z. To construct a transition system M for the system, an easy idea is to explore execution paths, and then construction the transition system M based on the explored paths. However, there is a shortcoming, that is, if a system consists of a lot of branches, a large number of execution paths will be explored, which will significantly slow down the performance of BMC. To efficiently verify a sequential system which holds a lot of branches, in BMC the single static assignment (SSA) technique [18] is employed to combine branches before applying BMC to carry out verification. This idea has been stated in paper [11]. E.g, as shown in Fig. 2.6, for the system shown in Fig. 2.5, we can firstly applied SSA technique to eliminate the branch, and then use the principle of BMC to construct a transition system M for the system.

In addition to the branches, a sequential system usually consists of loops. Unfortunately, the satisfiability modulo theory based SMT solver does not support the repetition structure (SAT solver also does not support repetition structure). As to successfully employ SMT solver to verify the system which holds loops, in BMC the loop within the system will be firstly unfolded as branch structure according to a loop bound. Particularly, in order to judge whether the loop is unfolded enough or not, an assertion which hold the negative condition of the loop is inserted into the end of the unfolding loop. E.g., for the given example shown in Fig. 2.7, the loop has been unfolded as branch structure when the loop bound is set to 2. Based on the unfolding process, BMC then can verify the system. Actually, in general BMC, there exist two types of bounds, one is the depth bound which is used to limit the search space, and the other is loop bound which is used to unfold loops. Based on the SMT-based BMC technique, a tool named

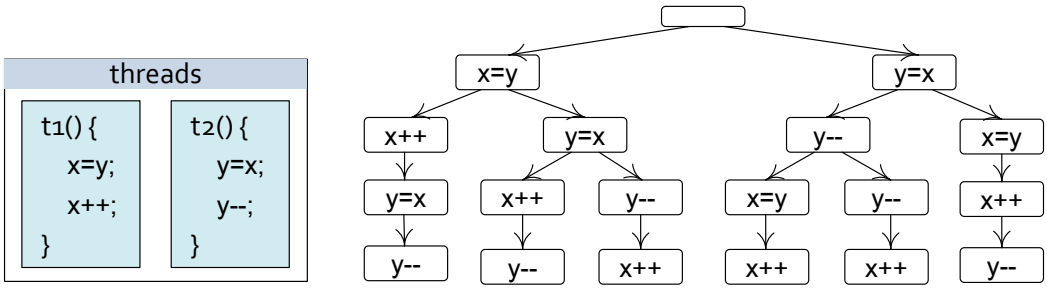

Figure 2.8: A concurrent ANSI-C system and its reachability tree

`cbmc` [16] has been established for the sequential C software.

In contrast with sequential software, the checking process of applying BMC to verify a concurrent system is more complex because of the scheduling behaviours. E.g., there is a ANSI-C concurrent system in Fig. 2.8. According to the ANSI-C standard, which thread is to be run is arbitrarily determined by ANSI-C scheduler. Usually, we named this type of scheduler as non-deterministic scheduler. Due to the non-deterministic scheduling behaviours, the running thread cannot be explicitly determined in the verification. In BMC, as to completely verify such system, all of the possible interleavings of threads will be taken into account in the checking process. In general, a tree named reachability tree is first constructed to demonstrate all of the interleavings, and then the constructed reachability tree is translated into the corresponding transition system M. E.g., for the given example shown in Fig. 2.8, the reachability tree illustrated in the right side of Fig. 2.8 can be used to demonstrate all of the interleavings of threads t_1 and t_2. According to the above presentation, BMC can exhaustively verify a concurrent system. Based on the SMT-based BMC technique, a tool named `esbmc` has been established for the multi-threaded software that conforms to ANSI-C standard.

2.3 Advantage and Disadvantage of Model Checking

Currently, model checking as an efficient checking technique has been widely applied to verify concurrent software. Moreover, with the development of model checking such as the improvement of automatization and application of advanced searching techniques, it has become an important checking technique in the verification of software. Compared with other checking techniques such as testing technique and theorem proving technique, model checking technique holds the

following advantages and disadvantages.

2.3.1 Advantages of Model Checking

Compared with testing technique, there are several advantages in model checking technique.

- If we apply testing technique to check a concurrent software, it is difficult to reproduce the error track although it can successfully find the error. However, based on the counterexample that is reported by model checking technique, we can easily constructed an error track for the detected bug and the counterexample clearly shows why the specification does not hold.

- Compared with testing technique, model checking supports temporal logical such as LTL in the verification of software, it can easily express many of the properties that are needed for reasoning about concurrent systems.

- If we apply testing technique to check a software which holds non-deterministic behaviours, it is difficult to completely check the software because of incomplete test-case set. As Edsger Dijkstra said: program testing can at best show the presence of errors but never their absence. However, based on the advanced satisfiability modulo theories (SMT), the SMT-based model checking can efficiently and completely verify the software which holds non-deterministic behaviours.

- If we apply testing technique to check a software which holds unbounded loops, it is difficult to judge whether the software satisfies a given property or not because of infinite states. However, based on the advanced k-induction and invariants method, model checking technique can successfully verify the software which holds unbounded loops.

In addition, compared with other formal verification techniques such as theorem proving, there are several advantages in model checking technique.

- Model checking is usually an automatic verification technique. Based on the different model checking techniques, there are many corresponding tools have been implemented such as `spin`, `cbmc` and `SMV`. Compared with theorem proving, in the verification process user does not need to construct a correctness proof.

- Compared with theorem proving, model checking technique is more efficient and scalable. In practice, as Clarke said in book titled "25 years of model checking": the other formal verification techniques may require months of time to prove a system.

2.3.2 Disadvantages of Model Checking

However, there is an important disadvantage that need to be solved or overcome in model checking technique, namely state space explosion problem. This is because, for a complex software, as to describe the all of the possible executions of the software, in model checking a huge checking model will be constructed in the verification process, which can easily make verification failed because of the memory or time limitations, although the advanced hardware has been used in our machines.

However, model checking still holds its own advantages in the development of software. E.g., for a complex software, we can firstly abstract a prototype model that holds the important behaviours of the target software rather than all of the details, and then apply model checking to verify the abstracted model with specification. Based on the verified model, developers can easily implement a more correct system. In addition, in the checking process we can applied model checking technique to check some key parts rather than whole software, e.g., the parts which hold non-deterministic behaviours.

Chapter 3

OSEK/VDX

OSEK/VDX, a standard of automobile OS promulgated by the alliance of German and French automotive manufacturers in 1994. The primary motivation of OSEK/VDX standard is to resolve the problem of increasing software content in vehicles and desire high-quality products. Currently, it has been widely adopted by many automobile manufacturers to design and develop a vehicle-mounted OS, such as BMW, Opel and TOYOTA. Based on the OSEK/VDX OS, more and more applications are developed and deployed in vehicles to assist drivers to control vehicles, such as the cruise control system and temperature control system. In this chapter, based on the specification of OSEK/VDX, we will briefly introduce OSEK/VDX OS and its applications.

3.1 OSEK/VDX OS

A general OSEK/VDX OS consists of a scheduler module, synchronization event process module, shared resource process module, alarm process module and interruption process module. Based on these system modules, OSEK/VDX OS supports a standardized application interfaces (APIs) for user to develop customized applications. In our research, we focus on the applications that communicate with scheduler module, synchronization event process module and resource process module. The structure of OSEK/VDX OS with an application is shown in Fig. 3.1.

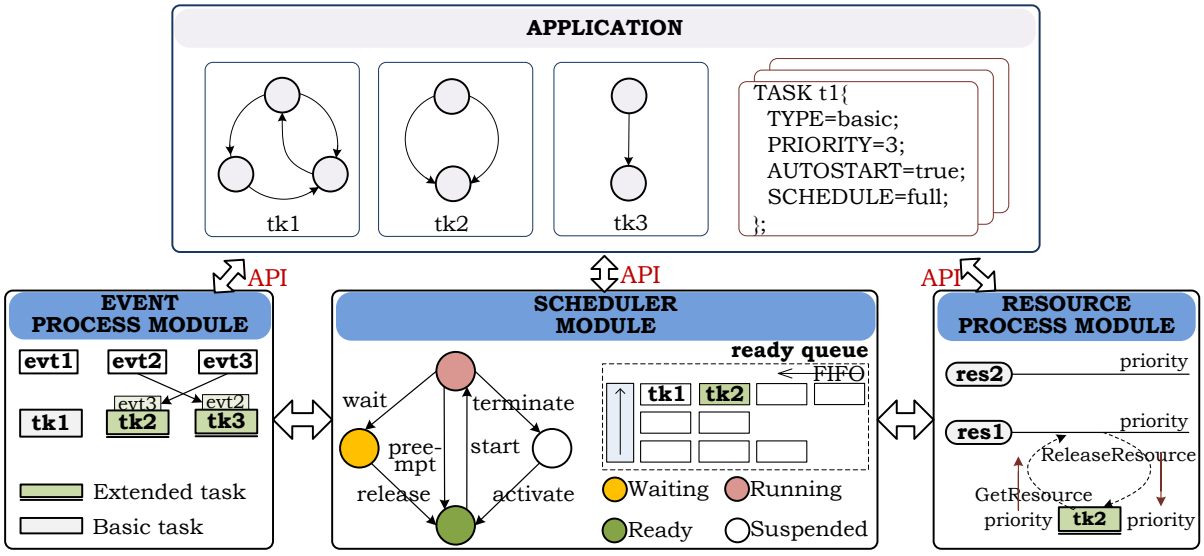

Figure 3.1: The structure of OSEK/VDX OS with an application

3.1.1 Scheduler Module

OSEK/VDX OS can process two types of tasks, basic task and extended task. The states of a basic task consist of *running* state, *suspended* state, and *ready* state. Compared with the basic task, the extended task can hold synchronization events and has a unique state called *waiting* state. In the scheduling process, the *static priority scheduling policy* with non-preemptive and full-preemptive strategies is adopted by the scheduler to conduct the executions of tasks. Particularly, in OSEK/VDX OS, scheduler manages a ready queue to lay out the execution order of tasks. Based on the ready queue, the running task is explicitly determined by OSEK/VDX scheduler.

In addition, scheduler module provides several APIs for applications, such as *TerminateTask*, *ActivateTask* and *ChainTask*. Tasks within the application can invoke these APIs to dynamically change the states of tasks, and the changed states will affect the scheduling of tasks, e.g., the context switch of tasks will happen when a suspended task is activated by running task using API *ActivateTask* or *ChainTask*. For instance, as shown in Fig. 3.2, there are two tasks $t1$ and $t2$, and the priority of task $t1$ is lesser than $t2$. Currently, task $t1$ is running task. When the API *ActivateTask*($tk2$) is invoked by $t1$, scheduler will move task $t2$ from *suspended* state to *ready* state. At this moment, the context switch of tasks will happen, because the priority

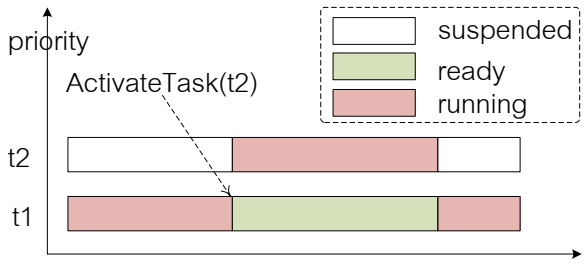

Figure 3.2: The context switch of tasks and API *ActivateTask*

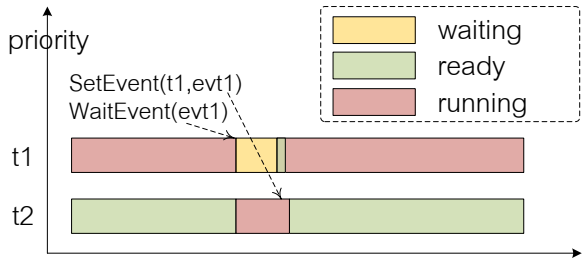

Figure 3.3: The synchronous behaviours between tasks

of task $t2$ is higher than $t1$.

3.1.2 Event Process Module

In the synchronization event process module, OSEK/VDX OS provides a synchronization mechanism for implementing synchronous executions between tasks. Particularly, only extended tasks can hold a definite number of events, and events are the criteria for the switching of task states from *running* state to *waiting* state or from *waiting* state to *ready* state. There are three APIs $SetEvent$, $WaitEvent$ and $ClearEvent$ that can be responded by event process module, and tasks within application can invoke these APIs to implement the synchronous behaviours. E.g., as shown in Fig. 3.3, when the running task $t1$ waits for the event $evt1$ using API $WaitEvent(evt1)$, task $t1$ cannot continue until the event $evt1$ is set by other tasks (basic tasks or extended tasks) using API $SetEvent(t1,evt1)$.

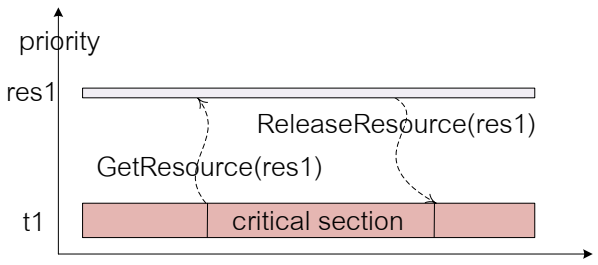

Figure 3.4: The priority ceiling protocol

3.1.3 Resource Process Module

OSEK/VDX OS adopts the *Priority Ceiling Protocol* [9] to coordinate the behaviors of task accessing shared resources in the resource process module. The resource process module supports two APIs (*GetResource* and *ReleaseResource*) that can be invoked by tasks to access shared resources. E.g., as shown in Fig. 3.4, if the API *GetResource*($res1$) is invoked by *running* task $t1$, and the priority of the task is lower than the ceiling priority of the resource $res1$, the priority of the task will be raised to the ceiling priority of the resource $res1$, and the priority of the task will be reset to the priority before requiring the resource $res1$ when *ReleaseResource*($res1$) is invoked by the task. Note that, the ceiling priority of a shared resource is lower than the lowest priority of all tasks that do not access the resource, and it is higher than the priorities of all tasks that access the resource.

3.2 OSEK/VDX Application and Execution Characteristics

3.2.1 OSEK/VDX Application

An application developed based on OSEK/VDX OS consists of two files, one is the source file, and the other is the configuration file. The source file which can be developed by C language is used to present the concrete behaviors of the application. The configuration file is used to define tasks, synchronization events and shared resources.

In the configuration file, the attribute `AUTOSTART` is used to set the initial state of tasks.

Source file (.cpp)

```
int a, b, c, h;

TASK t1(){
  if(b >= a){
    a++;
    ActivateTask(t2);
  }
  else{
    b++;
    ActivateTask(t3);
  }
  ActivateTask(t4);
  TerminateTask();
}
TASK t2(){
  if(h > 0)
    h--;
  else
    h++;
  TerminateTask();
}
TASK t3(){
  c=b;
  c=(c+b)/b;
  TerminateTask();
}
TASK t4(){
  assert(a > b);
  TerminateTask();
}
```

Configuration file (.oil)

```
TASK t1{
  TYPE=BASIC;
  SCHEDULE=FULL;
  PRIORITY=5;
  AUTOSTART=TRUE;
};

TASK t2
{
  TYPE=BASIC;
  SCHEDULE=NON;
  PRIORITY=6;
  AUTOSTART=FALSE;
};

TASK t3
{
  TYPE=BASIC;
  SCHEDULE=NON;
  PRIORITY=4;
  AUTOSTART=FALSE;
};

TASK t4
{
  TYPE=BASIC;
  SCHEDULE=NON;
  PRIORITY=4;
  AUTOSTART=FALSE;
};
```

Figure 3.5: An OSEK/VDX application

If the attribute `AUTOSTART` of a task is set to be `TRUE`, the task starts from *ready* state in the initial state (it will be inserted into ready queue according to the priority of the task). Otherwise, the task starts from *suspended* state. The attribute `SCHEDULE` is used to indicate the scheduling type. If the attribute `SCHEDUL` of a task is set to be `FULL`, the task can be preempted by higher priority tasks. Otherwise, the task cannot be preempted by higher priority tasks. A simple OSEK/VDX application without synchronization events and shared resources is shown in Fig. 3.5. As to clearly comprehend the execution characteristics of OSEK/VDX applications, an example is symbolically executed in this part.

In the application shown in Fig. 3.5, only the attribute `AUTOSTART` of $t1$ is set to be `TRUE`. Thus, $t1$ will be firstly moved to *running* state by scheduler and then executed in the initial

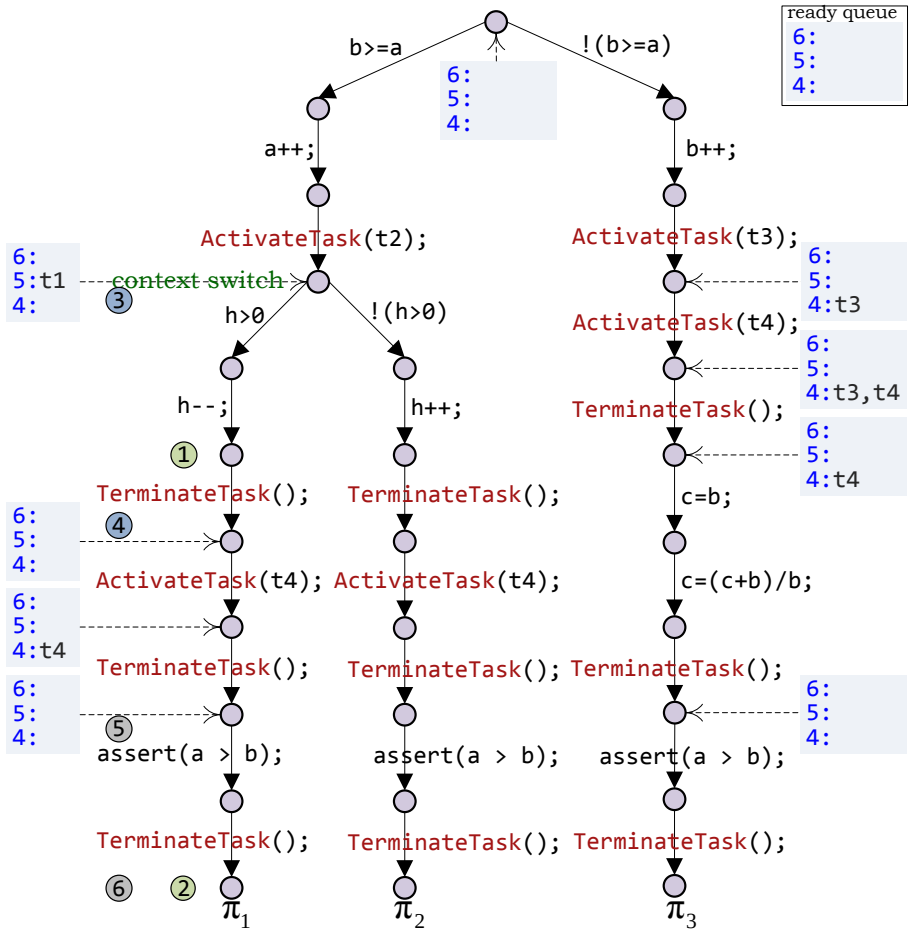

Figure 3.6: The execution tree for the application shown in Fig. 3.5

state. As shown in Fig. 3.6, when the API $ActivateTask(t2)$ is invoked by $t1$ (the branch "$(b >= a)$" is chosen to symbolically execute), scheduler will be loaded to respond to the API (task $t2$ is moved from *suspended* state to *ready* state). At this moment, the running task $t1$ will be preempted by $t2$ since the priority of $t2$ is higher than $t1$ and the attribute `SCHEDUL` of $t1$ is set to be `FULL` (the context switch happens after the API $ActivateTask(t2)$). Currently, task $t2$ gets run-unit to run, and goes to *suspended* state when the API $TerminateTask()$ is invoked ($TerminateTask()$ is used to terminate the executions of a task, and the terminated task will be moved from *running* state to *suspended* state by scheduler. If the running task is terminated, scheduler then dispatches the head task in the ready queue to run when ready queue is not empty). When task $t2$ terminates itself using API $TerminateTask()$, scheduler will dispatch task $t1$ to *running* state. Then, $t1$ continues its executions from the preempted point, and task

$t4$ is activated by $t1$ using API $ActivateTask(t4)$ (the context switch will not happen, since the priority of task $t4$ is less than $t1$). Finally, task $t4$ will be executed when the *running* task $t1$ terminates itself using API $TerminateTask()$. Thus, according to the conducted symbolic executions, we can get a task execution sequence $\langle t1, t2, t1, t4\rangle$ in the execution path π_1 and π_2. However, if the API $ActivateTask(t3)$ is invoked by task $t1$ (the "branch $!(b >= a)$" is chosen to symbolically execute), we will get a different task execution sequence $\langle t1, t3, t4\rangle$ in the execution path π_3.

3.2.2 Execution Characteristics of OSEK/VDX Application

Based on the execution tree of the application shown in Fig. 3.5, we can easily find the following execution characteristics of OSEK/VDX applications,

- Which task is to be run is explicitly determined by OSEK/VDX scheduler according to the ready queue and configuration data of application.
- The APIs invoked from tasks will haphazardly change the scheduling of tasks, e.g., the different service APIs locating at different branches will lead to different task execution sequences, and the context switch of tasks may happen when an API is invoked by running task.

According to the listed execution characteristics, there are several challenges that should be addressed when we apply model checking technique to verify an OSEK/VDX application, e.g.,

- How to precisely construct a model to represent the executions of the application.
- How to deal with OSEK/VDX OS, e.g., how to explicitly perform the scheduling behaviours of OSEK/VDX OS, and how to handle the APIs invoked from tasks.

We have proposed three approaches to verify OSEK/VDX applications in this thesis. The overview of the proposed approaches will be demonstrated in the next chapter.

Chapter 4

Overview of Checking Approaches

Based on the execution characteristics of OSEK/VDX applications, we have found, the running task within application is explicitly determined by OSEK/VDX scheduler (deterministic scheduling policy is used to dispatch tasks), and the APIs invoked from tasks will haphazardly affect the scheduling of tasks, e.g., (*i*) the context switch of tasks may happen when an API is invoked, (*ii*) the different APIs locating at different branches will lead to different task execution order. Although there exist many model checking methods [26][50][51] that have been applied to verify concurrent software such as the ANSI-C multi-threaded software and SystemC multi-threaded software, these methods usually perform an approximate verification because the behaviours of scheduler are not taken into account in verification process. If we apply these existing model checking methods to verify OSEK/VDX applications, a lot of unnecessary interleavings of tasks will be superfluously checked in the verification stage, especially these unnecessary interleavings will usually result in a spurious bug. Due to the spurious bug, developers have to spend extra costs judging whether the detected bug is real one or not after completing verification.

In addition, to accurately check concurrent software such as ANSI-C multi-threaded software and SystemC multi-threaded software, several prominent model checking methods [13][37] have been proposed by some senior researchers. In these methods, as to seek a more accurate verification result, the behaviours of scheduler are taken into account in the verification process. Even so, these prominent model checking methods are still unsuitable to accurately verify OSEK/VDX applications because these methods focus on the non-deterministic scheduler based multi-threaded software. In contrast with non-deterministic scheduler, in OSEK/VDX applica-

tions a deterministic scheduler is used to dispatch tasks. As to reduce the checking costs on the OSEK/VDX applications, a more accurate model checking approach should be proposed and applied in the verification process. In this thesis, we describe and develop three approaches that can accurately and automatically verify the safety property of OSEK/VDX applications using model checking technique.

4.1 Key Idea of Checking Approaches

The core of model checking consists of two processes, one is to construct a model for target system, and the other is to search the constructed model with given property. As to make model checking accurately verify OSEK/VDX applications, there are two ways that can be considered in the verification of OSEK/VDX applications, e.g.,

1. We can construct an accurate model for target OSEK/VDX application, and then apply existing searching algorithm to check the constructed checking model.

2. We do not desire an accurate model, but reform the existing searching algorithms to make the existing searching algorithms perform an accurate search in the verification process.

In our approaches, we will construct an accurate checking model to achieve the accurate verification based on the model checking technique. Compared with the method for reforming searching algorithms, there is an advantage in this method, e.g., once an accurate model is constructed, we can directly use different searching algorithms to accurately verify OSEK/VDX applications. As we know, in the verification of software, we usually should use different model checking techniques to exhaustively find bugs. Compared with the method for constructing accurate model, the method for reforming the existing searching algorithms will usually spend more costs reforming the different searching algorithms.

To construct an accurate model for a target OSEK/VDX application, the behaviours of OSEK/VDX OS such as scheduler behaviours should be taken into account in the constructing model process. This is because, when an OSEK/VDX application runs on the OSEK/VDX OS, the running task is explicitly determined by OSEK/VDX scheduler and the APIs invoked from tasks will haphazardly affect the scheduling of tasks. Based on this idea, we have proposed three

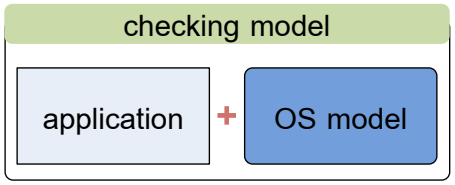

Figure 4.1: The key idea of Spin-based checking approach

approaches that can accurately and automatically verify the developed OSEK/VDX applications using model checking techniques.

4.2 Overview of Checking Approaches

In our first approach, we investigate a method based on the existing model checker Spin. In the method, as to make Spin model checker accurately verify OSEK/VDX applications, a checking model which is a combination model of application model and OSEK/VDX OS model is employed to precisely simulate the executions of the OSEK/VDX applications in real OSEK/VDX OS. In the second approach, as to verify a complex application which holds a large number of states, we develop a new technique named execution path generator (EPG) to verify OSEK/VDX applications based on the advanced SMT-based bounded model checking technique. Particularly, in the approach the OSEK/VDX OS model is embedded in EPG (the algorithm of constructing model flat) to respond to the invoked APIs and determine running task in the process of constructing application model. In the last approach, we present a novel method to translate OSEK/VDX applications into sequential models in order to apply the existing model checkers such as `spin` and `cbmc` to efficiently verify OSEK/VDX applications.

4.2.1 Spin-based Checking Approach

To accurately verify OSEK/VDX applications using model checking technique, a cheap method is based on the existing model checker, because we do not need to establish a new model checker. As we know, it is too difficult and impractical to establish a new model checker within a short time, since a basic model checker usually consists of several complex modules such as the language parsing, simulation, verification, counterexample generation and display. To finish

all of these modules with reasonable soundness guarantee often takes years of effort, e.g., the established model checkers `spin` and `cbmc` are the result of decades of development. As to apply existing model checkers to accurately verify OSEK/VDX applications, we investigate an approach based on the well-know Spin model checker.

To accurately verify OSEK/VDX applications using Spin model checker, the key work is how to construct an accurate checking model to feed Spin. In our Spin-based checking approach, as shown in Fig. 4.1, we design a checking model which is a combination model of application model and OSEK/VDX OS model to precisely simulate the executions of the target application. In the combination model, all of the tasks within application are designed as `process`es, and the OSEK/VDX OS model as a cooperative `process` is used to dispatch tasks and respond to the APIs invoked from tasks. In particular, the application model and OSEK/VDX OS model are synchronously executed based on the invoked APIs for simulating the executions of the application in real OSEK/VDX OS.

The advantage of the approach is that, it can accurately verify OSEK/VDX applications based on the existing model checker Spin. However, the scalability of this approach is limited, because too many details of OSEK/VDX OS model will be explored in the verification stage, especially the state space explosion may happen if application invokes a lot of APIs (when an API is invoked by running task, OSEK/VDX OS model will be checked once by Spin in the verification process). Based on the shortcoming of the approach, we can easily find that the approach cannot be applied to verify the complex OSEK/VDX applications which will invokes a lot of APIs. In order to efficiently check a complex OSEK/VDX application which holds a lot of states and APIs, we develop a new technique named execution path generator (EPG) based on the advanced SMT-based bounded model checking.

4.2.2 SMT-based Bounded Model Checking Approach

The core of our SMT-based bounded model checking approach is execution path generator (EPG), which is a new technique to verify OSEK/VDX applications using bounded model checking technique. In our EPG technique, as to efficiently handle a complex application which holds a large number of states, the advanced SMT-based bounded model checking is adopted to perform the verification. Particularly, in order to avoid OSEK/VDX OS model to be explored in the

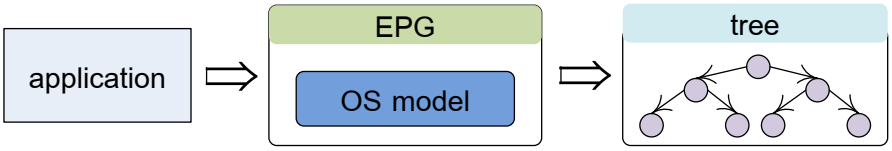

Figure 4.2: The key idea of SMT-based BMC approach

verification stage, as shown in Fig. 4.2, we embed OSEK/VDX OS model in EPG (the algorithm of constructing model flat) to dispatch task and respond to the APIs invoked from tasks.

To verify an OSEK/VDX application using bounded model checking technique, the key work is how to construct a transition system for the application. In OSEK/VDX applications, the APIs invoked from tasks will haphazardly affect the scheduling of tasks, e.g., the context switch of tasks may happen when an API is invoked, and the different APIs location at different branches will lead to different task execution orders. In our EPG technique, like other bounded model checking for concurrent software (e.g., Lucas' method [37] for ANSI-C multi-threaded software), a reachability tree as an intermediate form is constructed for establishing the transition system of application.

The advantage of the approach is that, it is more efficient and scalability than the Spin-based checking approach. This is because, in EPG technique the state-of-the-art SMT-based bounded model checking is employed to perform the verification, which make EPG technique more efficient. Moreover, in EPG technique the OSEK/VDX OS model will not be explored in the verification stage since the OSEK/VDX OS model is embedded in EPG to dispatch tasks and respond to the APIs invoked from tasks, which make EPG technique more scalable. However, we can easily find that the approach is not efficient to check the large-scale applications which hold a lot of branches, because EPG will spend a lot of time exploring execution paths for constructing the reachability tree in the checking process, which will slow down the efficiency of the approach. As to efficiently handle the applications which hold a large number of branches, we propose a novel approach based on the sequentialization idea.

4.2.3 Sequentialization-based Checking Approach

The key idea of our sequentialization-based checking approach is that, we firstly translate a given OSEK/VDX application into a corresponding sequential model, and then employ the

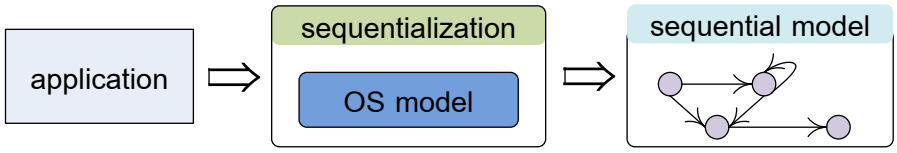

Figure 4.3: The key idea of sequentialization-based checking approach

existing model checker such as `spin` and `cbmc` to verify the sequential model. According to the shortcoming of EPG technique, we have found that EPG technique is not efficient to verify the large-scale applications which hold a lot of branches, because it will spend a lot of time exploring execution paths for establishing transition system. As to efficiently verify the applications which hold a lot of branches, the core work is how to efficiently construct a sequential model for the target application. In our sequentialization-based checking approach, we use an extended directed graph instead of reachability tree to construct the sequential model of OSEK/VDX application. Particularly, as to avoid the behaviours of OSEK/VDX OS to be poured into the sequential model, as shown in Fig. 4.3, like EPG technique we embed OSEK/VDX OS model in translation algorithm flat to dispatch tasks and respond to the invoked APIs. The main process of our sequentialization-based checking approach is that, we symbolically execute application using an extended directed graph and explicitly perform the scheduling behaviours when meeting an API in the sequentialization process.

There are several strengths from extended directed graph that can make our sequentialization-based checking approach more efficient than EPG technique in checking the applications which hold a lot of branches, e.g.,

- Based on the directed graph, we do not need to explore execution paths when meeting branches in the sequentialization process, instead, we can use the combination states to reduce the computation times. E.g., for the example shown in Fig. 4.4, if we use tree to describe the executions of the given example, we have to compute 12 times to construct the tree. However, compared with tree, we just compute 8 times when directed graph is used to describe the executions of the example.

- Based on the directed graph, we do not need to repeatedly compute the same execution behaviours of application in the sequentialization process, instead, we can construct a cycle in directed graph to represent the same execution behaviours. E.g., as shown Fig. 4.5,

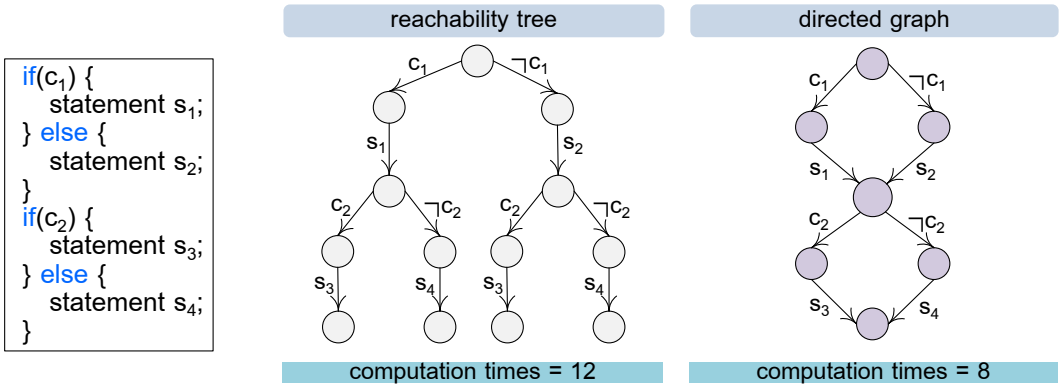

Figure 4.4: An example for tree and directed graph (branch)

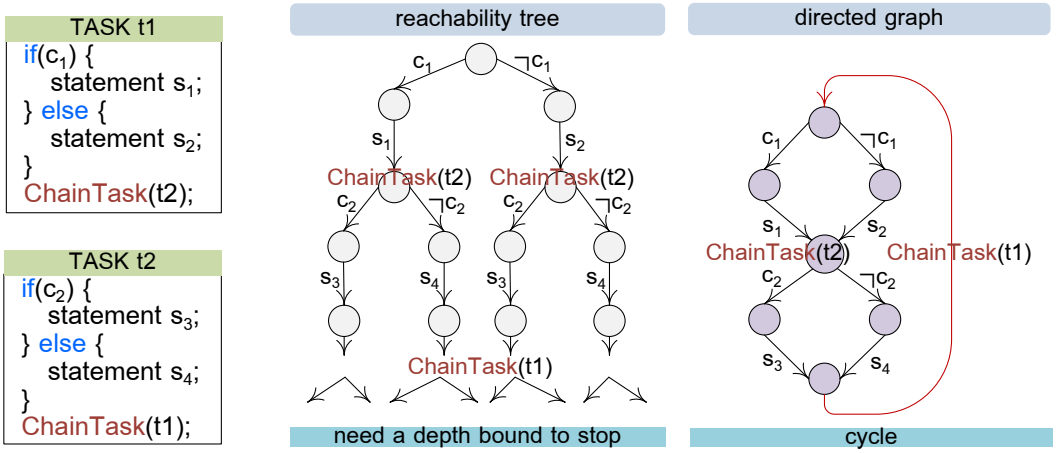

Figure 4.5: An example for tree and directed graph (loop)

there are two tasks $t1$ and $t2$ that will activate each other using API $ChainTask(tk)$. The example holds an infinite execution sequence $t1, t2, \cdots, t1, t2$. If we use tree to describe the executions of the example, we not only need a depth bound to stop the process of constructing tree, but also will spend a lot of computation times to construct the tree under the set depth bound. However, compared with tree, in directed graph we can use a cycle to represent the the same execution behaviours.

Furthermore, based on the sequentialization process of our approach, the existing model checker such as `spin` and `cbmc` can efficiently verify OSEK/VDX applications, because existing model checker just check a sequential model instead of a concurrent model.